

[1, 2) 上のベータ変換に基づく非再帰型 n ビット 擬似乱数の生成 ($n = 64, 128, 192, \dots, 8192$)

(2025 年春学会アブストラクト再編集)

1 非再帰型 n ビット擬似乱数の生成と検定 ($n = 64, 128, 192, \dots, 8192$)

以下の (A0)-(A5) からなるアルゴリズムで乱数列を生成する：

(A0) n ビットの 2 整数 B, x を準備する。 B の最上位ビット (MSB, Most Significant Bit) は 1 とする。

(A1) x の MSB を 1 にする (大文字の X で表すことにする)。

(A2) 2 数 B, X を掛けて、 $2n$ ビットの数 $\gamma \equiv B \cdot X$ を得る。

(A3) $\gamma \equiv B \cdot X$ のちょうど真ん中にある連続する n ビットを取り出し、それを新たな x とする。

(A4) (A1) から (A3) を 5 回繰返し ($Lp = 5$ で表す), 最後の γ から取り出した n ビット数 x を、 1 つの乱数値 $z(\equiv x)$ とする。

(A5) (A0) の B を $B_0, B_1, B_2, B_3, \dots$ と色々変化させて ((A0) の x は固定値でよい), 各 B_i ごとに (A0) から (A4) を行なって、多くの乱数値 $z_0, z_1, z_2, z_3, \dots$ を得る。
(非再帰型の乱数生成) ($n=64, 128, 192, 256, 384, 512, 1024, 2048, 4096, 8192$)

[以下、引用を簡単にするため、このアルゴリズムを「採中乗算」(extract Middle and Multiply, xMM) アルゴリズムと呼ぶことにする]

(A0) の x (固定値) および (A5) の B_i は以下の様にとる。 $e = 2.7182\dots$ を自然対数の底とし、

$(1 + \frac{e}{10}) \times 8 = 10.1746254\dots = (16 \text{ 進表記}) 0\text{xa.}2\text{cb}4411\text{ ba}257552\text{ 232c}1139\dots$
の左から n ビットを x とする。例えば $n = 64$ の場合、 $x = 0\text{xa}2\text{cb}4411\text{ba}257552$ である。 B_i は、 i を 64 ビットで表し (ただし $n = 64$ の時は 48 ビットで表す), 上述の x と丁度真ん中位置で XOR (排他的論理和, $\oplus$) をとったものとする。例えば $n = 64$ の場合、 $i = 3$ のときは $(\text{a}2\text{cb}4411\text{:ba}257552) \oplus (0000000\text{:}0000003)$ となるので $B_3 = 0\text{xa}2\text{cb}4411\text{ba}257652$ となる。(註: B_i, x は別の方法で定めてもよい。ここでは $n = 8192$ の場合に $\pi - 2.5 = 0.64159\dots = 0\text{x}0.\text{a}43\text{f}6\text{a}\dots$ を使って決めている。)

n ビット数列 $\{z_i\}_i$ の乱数性の検定 は以下のそれぞれの場合について行う：

[検定対象 a] n ビット全部, [検定対象 b] n ビット中の最初の 64 ビットのみ。

[NIST の sts-2.1.2 による乱数性のテスト] 1000000 ビットからなるストリーム 1000 本分を用意したファイルを作成し、全ての検定項目についてテストを行う。(テスト結

果として 188 個の p 値と PROPORTION の組が生成される。検定時のパラメーターは Block Frequency Test の block length を 20000 に変更し、その他は規定値のまま。) この全検定を繰返し 3 回行い、同一のテスト項目について、2 回以上範囲外のテスト値が出た場合そのテストに不合格と判断することとする。結果は、全ての $n = 64, 128, \dots, 4096, 8192$ について [検定対象 a],[検定対象 b] とともに合格である。

[L'Ecuyer & Simard の TestU01 による乱数性のテスト] $n = 64, 128, \dots, 4096$ については battery Crush の全項目 (96 のテスト項目がある) についてテストを行う。テスト結果である p 値が範囲 $[0.001, 0.9990]$ の外側に出た場合には、さらに 3 回テストを追加し、2 回以上範囲外の p 値が出た場合そのテスト項目に不合格と判断することとする。 $n = 8192$ の場合は検定時間との兼ね合いで battery SmallCrush の全項目についてテストを行う。結果は、全ての n について [検定対象 a],[検定対象 b] とともに合格である。(註: $n = 4096$ の場合、[検定対象 b] のテストでは延べひと月程度の時間がかかる。)

2 乱数生成アルゴリズムの $[1, 2)$ 上の β 変換による記述

$\beta > 1$ とし $[1, 2)$ 上の β 変換 $T_\beta(X) = \beta X - \lfloor \beta X \rfloor + 1$, $X \in [1, 2)$, を考える ($\lfloor q \rfloor$ は q の整数部分を表す)。ここでは、さらに

β が $\beta = 2^k B$, $B \in [1, 2)$, と表されている場合を考える。

(記号 B, X は §1 の B, X と敢えて混用。) $\beta X \equiv 2^k BX$ 中の BX を 2 進表現で $BX = w_0 w_1 \cdot w_2 w_3 w_4 \dots$ とすると (註: $BX \in [1, 2)$ のとき $w_0 w_1 = 01$, $BX \in [2, 4)$ のとき $w_0 = 1$) , $2^k BX$ は小数点を基準位置として BX の k ビット左シフトとなる, すなわち,

$$BX = w_0 w_1 \cdot w_2 w_3 w_4 \dots \implies 2^k BX = w_0 w_1 \dots w_{k+1} \cdot w_{k+2} w_{k+3} \dots .$$

そして $T_\beta(X)$ では整数部分の $w_0 \dots w_{k+1}$ が 1 となるので

$$T_\beta(X) \equiv T_{2^k B}(X) = 1.w_{k+2} w_{k+3} \dots \quad \text{となる。}$$

通常計算機では数値に割り当てるビット数は予め決めてあるので (例えば整数で n ビット, その乗算結果は $2n$ ビット), B, X として n ビットの $B \equiv 1.u_1 \dots u_{n-1}$ と $X \equiv 1.v_1 \dots v_{n-1}$ を割り当てるとすれば, 乗算 BX の結果は $2n$ ビットの $BX \equiv w_0 w_1 \cdot w_2 w_3 \dots w_{2n-1}$ となる。すると, $(2^k)BX = w_0 w_1 w_2 \dots w_k w_{k+1} \cdot w_{k+2} w_{k+3} \dots w_{2n-1}$ であるので, $T_\beta(X)$ は $T_{2^k B}(X) = 1.w_{k+2} w_{k+3} \dots w_{2n-1}$ となり $(2n-1) - k$ ビット値である。この $T_\beta(X)$ が n ビット超の時は, 次の乗算の変数として使うため n ビットに切り詰められる ($\rightarrow$ 丸め誤差 rounding error の発生), すなわち,

$$T_{2^k B}(X) = 1.w_{k+2} w_{k+3} \dots w_{2n-1} \implies 1.w_{k+2} w_{k+3} \dots w_{k+n} \quad (n \text{ ビットに切り詰め})。$$

上式で特に $k = (n/2) - 1$ ととれば, $T_{2^{(n/2)-1} B}(X)$ は $(3/2)n$ ビットであり, この $T_{2^{(n/2)-1} B}(X)$ の後ろ $\frac{n}{2}$ ビットをカットして n ビットに切り詰めた結果は,

$T_{2^{(n/2)-1} B}(X) = 1.w_{(n/2)+1} w_{(n/2)+2} \dots w_{2n-1} \implies 1.w_{(n/2)+1} w_{(n/2)+2} \dots w_{(3n/2)-1}$ (n ビット切詰) となり, $2n$ ビットの $BX \equiv w_0 w_1 \cdot w_2 w_3 \dots w_{2n-1}$ から丁度真ん中の n ビット $w_{(n/2)}$

$w_{(n/2)+1} \cdots w_{(3n/2)-1}$ を取り出して $1 \cdot w_{(n/2)+1} \cdots w_{(3n/2)-1}$ としたものになっている。

n ビット変数 B, X に対する上述 ($k = (n/2) - 1$) の計算機上の計算を、小数点を取り除いて眺めてみると、MSB = 1 の 2 数 B, X を掛けて、その結果から、真ん中の n ビット $w_{n/2}w_{(n/2)+1} \cdots w_{(3n/2)-1}$ を取り出して (=頭の $n/2$ ビットと尻の $n/2$ ビットを落として)、その MSB である $w_{n/2}$ を 1 にしていることが分かる。これらは § 1 のアルゴリズム (A2),(A3),(A1) に対応している。アルゴリズムの (A4) は、このビット長が n ビット (乗算結果は $2n$ ビット) に固定された $[1,2)$ 上の β 変換を 5 回繰返すことに相当している。以上のように変数のビット長が n ビットに制限された $[1,2)$ 上の β 変換を、便宜上、「**ビット長固定型**」の離散 β 変換と言うことにすれば、§ 1 のアルゴリズム xMM は「**ビット長固定型**」の $[1,2)$ 上の離散 β 変換で $k = (n/2) - 1$ とした場合の繰返しとして記述されることとなる (最後の MSB 値は 1 にしないでそのまま使う)。

3 「**ビット長非固定型**」離散 β 変換とエルゴード理論の β 変換

エルゴード理論でよく現れる β 変換は $\Phi_\beta(t) = \beta t - [\beta t], t \in [0, 1), \beta > 1$, であり、その発展形である linear mod 1 変換は $\Phi_{\beta,\alpha}(t) = \beta t + \alpha - [\beta t + \alpha], t \in [0, 1), \beta > 1, \alpha \in (0, 1)$, である。いずれも $[0,1)$ 上の (β) 変換である。一方、ここで扱っている $[1,2)$ 上の β 変換 $T_\beta(X) = \beta X - [\beta X] + 1, X \in [1, 2)$, は、

$T_\beta(X) = \beta X - [\beta X] + 1 = \{\beta(X - 1) + (\beta - [\beta])\} - [\beta(X - 1) + (\beta - [\beta])] + 1$ と変形されるので、 T_β は $\alpha = (\beta - [\beta])$ とした linear mod 1 変換 $\Phi_{\beta,\alpha}(t) = \beta t + \alpha - [\beta t + \alpha]$ を X-軸方向に 1, Y-軸方向に 1 平行移動したのになっている。このことは § 1 で記述した単純なアルゴリズムで、なぜ乱数性が発生するかを考察する際に、エルゴード理論の β 変換の理論からの援用が期待できる可能性を示している。しかしながら、上述の § 1 (A3) では、 n ビットの範囲では扱うことの出来ない最下位ビットを含む過剰ビットを捨ててしまう操作 (丸め操作) が行われている。この丸め操作はエルゴード理論での β 変換の理論の中では扱いにくい。そこで、§ 1 の (A3) で丸め操作を行なわないでそのまま計算を続ける ($\Rightarrow$ 本来の β 変換のアルゴリズムをそのまま使う) 「**ビット長非固定型**」の離散 β 変換計算を行えばエルゴード理論での β 変換と同じ扱いができる。

具体的には、§ 1 の (A3) で、真ん中にある連続する n ビットのみを取り出す操作の代わりに、この n ビットを含む以下全部のビットを使って計算を続けることになる。(この計算はビット長が非固定であるため、乗算を繰返すと計算ビット長が長くなっていく。) 最後の 2 数の乗算結果から頭の $n/2$ ビットを棄て、続く n ビットを取り出したもの ($\{\zeta_i\}_i$ と記す) が、「**ビット長固定型**」の乱数値 $\{z_i\}_i$ に対応する数値になる。

「**ビット長非固定型**」計算 T_β で得られた数値 $\{\zeta_i\}_i$ が、「**ビット長固定型**」計算 T_β から得られる $\{z_i\}_i$ と同等の乱数性を有していなければ意味がないので検定を行う。検定は $n = 64, 128, 192, 256, 384, 512$ の各 $\{\zeta_i\}_i$ に対して、TestU01 を「**ビット長固定型**」と同様に [検定対象 a],[検定対象 b] に適用した。結果は $n = 64$ の 1 つのテストを除いて

(§ 4 表参照) すべて合格である。この結果から、§ 1 で記述したアルゴリズムでなぜ乱数性が発生するかを考察する際に、 β 変換の理論からの援用が期待できることとなった。

4 「採中乗算 (xMM)」アルゴリズムの繰返し回数と乱数性

xMM アルゴリズムでは $T_\beta(X)$ の繰返し回数 Lp を 5 としているが、 $Lp = 5$ はいろいろな値を試した結果経験的に得られた値である。下表は、 $n = 64, 128, 192, 256$ の場合において、 $Lp = 3, 4, 5, 6$ に対する TestU01 ([検定対象 b]) の結果である。

採中乗算 [検定対象 b]	$n = 64$		$n = 128$		$n = 192$		$n = 256$	
	固定長	非固定長	固定長	非固定長	固定長	非固定長	固定長	非固定長
$Lp = 3$	$\uparrow 3 \downarrow 4 \downarrow 27$	$\uparrow 4 \downarrow 3 \downarrow 39$	$\uparrow 2 \downarrow 5 \downarrow 34$	$\uparrow 4 \downarrow 3 \downarrow 40$	$\downarrow 4 \downarrow 33$	$\uparrow 4 \downarrow 3 \downarrow 34$	$\uparrow 4 \downarrow 1 \downarrow 40$	$\uparrow 4 \downarrow 1 \downarrow 41$
$Lp = 4$	$\downarrow 2$	$\downarrow 4 \downarrow 14$	$\downarrow 1 \downarrow 3$	$\uparrow 3 \downarrow 2 \downarrow 14$	o	o	o	$\downarrow 2 \downarrow 2$
$Lp = 5$	o	$\downarrow 1$	o	o	o	o	o	o
$Lp = 6$	o	o	o	o	o	o	o	o

表中の $\uparrow e, \uparrow e, \downarrow e, \downarrow e$ は、それぞれ p 値が $1 - 10^{-15}$ 以上、 $1 - 10^{-300}$ 以上、 10^{-15} 以下、 10^{-300} 以下になったテストの個数が e であることを表す。o は全ての p 値が $\uparrow, \uparrow, \downarrow, \downarrow$ に該当しないことを表す。この表から「ビット長固定型」では n にかかわらず $Lp = 5$ が妥当な値であることが分かる。xMM (採中乗算) アルゴリズムでは、1 回の T_β ごとに乗算結果の頭 $n/2$ ビットが落ちるので、 $Lp = 5$ では延べ $(5/2)n = 2.5n$ ビットが落ちて n ビットの乱数が出現している。

5 乗算後の先頭 $n/4$ ビットを落とすアルゴリズムと繰返し回数

ここでは $T_{2^k B}(X)$ において $k = \frac{n}{4} - 1$ の場合を考える。これは 2 数 B, X の乗算結果 BX がなす $2n$ ビットから、頭の $\frac{n}{4}$ ビットを落とすことに相当する。下表は xMM と同様に Lp を変えたときの様子である。 $Lp = 9$ あたりから乱数性が出ている。1 回の

棄四半 [検定対象 b]	$n = 64$		$n = 128$		$n = 192$		$n = 256$	
	固定長	非固定長	固定長	非固定長	固定長	非固定長	固定長	非固定長
$Lp = 6$	$\uparrow 1 \downarrow 2 \downarrow 11$	$\uparrow 3 \downarrow 4 \downarrow 24$	$\downarrow 2 \downarrow 7$	$\downarrow 3 \downarrow 18$	$\downarrow 1 \downarrow 11$	$\uparrow 1 \downarrow 8 \downarrow 21$	$\uparrow 2 \downarrow 4 \downarrow 23$	$\uparrow 2 \downarrow 2 \downarrow 30$
$Lp = 7$	$\downarrow 3$	$\downarrow 1 \downarrow 4$	$\downarrow 1$	$\downarrow 1 \downarrow 5$	$\downarrow 1$	$\downarrow 2$	$\downarrow 1 \downarrow 1$	$\uparrow 1 \downarrow 3 \downarrow 9$
$Lp = 8$	$\downarrow 1$	$\downarrow 2$	o	$\downarrow 1$	o	o	o	$\downarrow 1$
$Lp = 9, 10$	o	o	o	o	o	o	o	o

T_β で $n/4$ ビットが落ちるので、 $Lp = 9$ では延べ $(9/4)n = 2.25n$ ビットが落ちている。

6 数理的な考察

単純なアルゴリズム xMM が、何故 n によらず $L_p = 5$ で乱数を生成するかを考察する。整数関数 I_m を

$$T_{2^k B}^m(X) = 2^k B \cdot T_{2^k B}^{m-1}(X) - I_m, \quad I_m = \lfloor 2^k B \cdot T_{2^k B}^{m-1}(X) \rfloor - 1$$

で定めれば、

$$T_{2^k B}^\ell(X) = (2^k B)^\ell X - (2^k B)^{\ell-1} I_1 - (2^k B)^{\ell-2} I_2 - \cdots - (2^k B)^2 I_{\ell-2} - (2^k B) I_{\ell-1} - I_\ell$$

と表されるので、右辺初項の X の係数 $(2^k B)^\ell$ ($\approx T_{2^k B}^\ell$ のグラフ中に現れる線 (枝) の数) に着目する。 B の最小変化は $\Delta \equiv 2^{-(n-1)}$ であるので $(2^k(B + \Delta))^\ell$ と $(2^k B)^\ell$ の差は

$$2^{k\ell} \{(B + \Delta)^\ell - B^\ell\} = 2^{k\ell} \{\ell B^{\ell-1} \Delta + \ell C_2 B^{\ell-2} \Delta^2 + \cdots + \ell B \Delta^{\ell-1} + \Delta^\ell\}$$

となり、 $\ell 2^{k\ell} B^{\ell-1} \Delta$ が違いの主要部分となる。 $R(k, \ell) \equiv 2^{k\ell} \Delta / 2^n = 2^{k\ell - (2n-1)}$ とおくと (ビット長 n によるある種の正規化)、 $R(\frac{n}{s} - 1, \ell) = 2^{(\ell/s - 2)n - \ell + 1}$ は $\ell = 2s$ を境に n の符号が $-$ から $+$ に転じている。 $\ell = 2s$ ($\Rightarrow$ 落ちる総ビット数 $= \frac{n}{s} \times 2s = 2n$) は「乱数性が出てくる転換点」と考えられる。考察の結論として

(i) 1 回あたり $K (\leq \frac{n}{2})$ ビット落とす T_β を L_p 回繰返して n ビット乱数が得られる時、 $K \times L_p > 2n$ を満たす、と考えられる。

(ii) 特に $K = \frac{n}{s}$ のとき、上不等式は $L_p > 2s$ と n に依存しない。

(iii) xMM は $L_p = 5 > 4 = 2s (s = 2)$ のビット長固定型離散 β 変換に対応したアルゴリズムである。

[参考文献] http://www.math.kobe-u.ac.jp/publications/yagsuppl/kakenDocument_2013.pdf